CS4475 Fall 2026
Kevin Pinon
Kpinon2@miners.utep.edu

Lab Report
Submitted on 9/23/26

PA 1: Implementing xv6 User Commands and System Calls

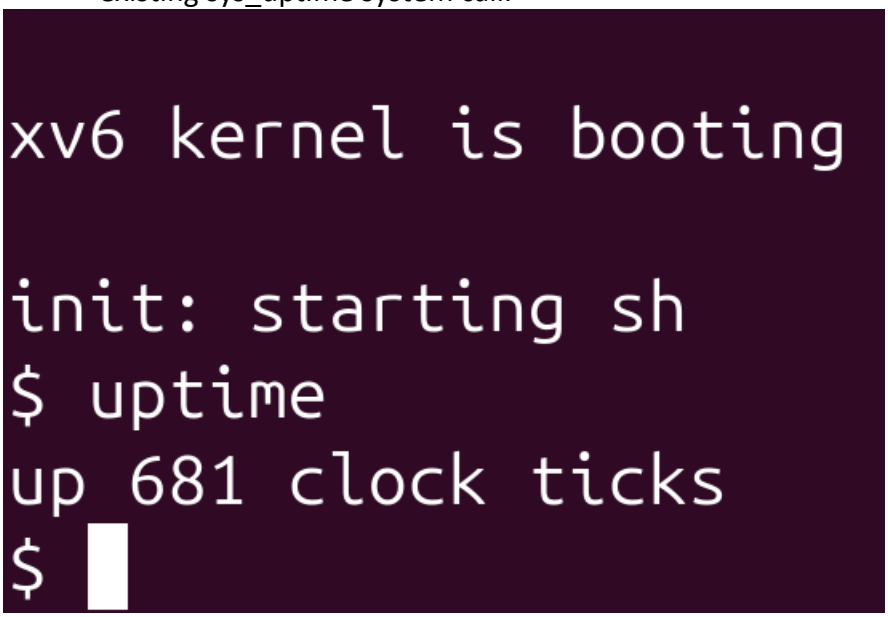
Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (5 pts) Implement the uptime utility
Files Changed/Created:

- User/uptime.c: Created a user-level program that calls the uptime() system call and prints the total system clock ticks using “printf(“up %d clock ticks\n”, ticks);”
- Makefile: Added $U/_uptime to the UPROGS list so the utility compiles and gets included in the xv6 filesystem image.

Purpose of uptime.c:

- It fetches kernel boot-time metrics by providing a command-line interface for the existing sys_uptime system call.



Task 2. (10 pts) Implement a time1 command that reports elapsed time.
Files Changed/Created:

- User/time1.c: Created a user program that records the system uptime via uptime() before and after running a child process using fork() and exec(). It uses standard wait() to block until the child finishes, calculates the difference between the end and start clock ticks and prints the total elapsed time.

- User/matmul.c &user/sleep.c: Added test programs given to us to test CPU usage and sleeping behavior
- Makefile: Registered $U/_time1, $/_matmul, and $U/_sleep in the UPROGS build target so they compile into the xv6 user environment

Purpose of time1:

- To measure total wall-clock execution time for user programs and background processes

```
$ time1 matmul
Time: 4 ticks
elapsed time: 4 ticks
$ time1 matmul &; time1 matmul&
$ Time: 7 ticks
elapsed time: 8 ticks
Time: 6 ticks
elapsed time: 7 ticks
time1 sleep 10
elapsed time: 10 ticks
$
```

Task 3. (10 pts) Keep track of how much cputime a process has used.

1. Kernel/proc.h
- Added “uint cputime;” to struct proc. Kernel can now store a running CPU counter of clock ticks.
2. Kernel/proc.c
- Inside allocproc() I added “p->cputime = 0;”. This allows the CPU counter process to start with 0.
3. Kernel/trap.c
- Inside usertrap() and kerneltrap() I added “myproc()->cputime++” under timer interrupt condition. With every interrupt the kernel increments the cputime counter by 1 tick.

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child’s status and rusage.

Files Changed/Created:

- Kernel/pstat.h: Created header file defining “struct rusage{uint cputime;};” to structure process resource usage stats passed to user space
- User/user.h: Added forward declaration “struct rusage;” and system call prototype “int wait2(int* , struct rusage*);”.
- User/usys.pl: Added “entry(“wait2”);” to generate the RISC-V assembly ecall wrapper for userland programs.

- Kernel/syscall.h: Defined system call number “#define SYS_wait2 23”.
- Kernel/syscall.c: Registered “[SYS_wait2] sys_wait2” in the kernel system call dispatch table
- Kernel/defs.h: Added prototype “int kwait2(uint64, uint64);” to make the kernel implementation visible across files.
- Kernel/sysproc.c: Implemented sys_wait2() to fetch user-space pointer arguments via argaddr() and pass them to kwait2().
- Kernel/proc.c: Implemented “kwiat2(uint64 addr, uint64 rusage_addr)”. It scans the process table for zombie children, reads their exit status and accumulated cputime and copies those values back into user-space memory using copyout() before cleaning up the child process.

Purpose of wait2():

- It extends the standard wait() functionality to allow parent processes to retrieve both the child’s exit status and its resource usage across the user-kernel space boundary.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

Files Changed/Created:

- User/time.c: Created the user utility that measures total elapsed time, CPU ticks, and CPU utilization percentage for an executed command. It records start time using uptime(), spawns the requested command via fork() and exec() and calls “wait2(&status, &ru)” to tetrieve the child process’s accumulated cputime. Upon child termination, it measures end time using uptime() and computes “% CPU = (cputime * 100) / elapsed_time”.
- Makefile: Registered $U/_time in UPROGS so the executable compiles into the xv6 filesystem image.

```
$ time matmul
Time: 4 ticks
elapsed time: 4 ticks, cpu time: 4 ticks, 100% CPU
$ time matmul &; time matmul &
$ Time: 7 ticks
elapsed time: 8 ticks, cpu time: 4 ticks, 50% CPU
Time: 6 ticks
elapsed time: 7 ticks, cpu time: 4 ticks, 57% CPU
time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

This assignment was my first experience with xv6 and operating system kernel concepts. Learning how the various components work together, like system call dispatching, timer interrupts, and memory management, was a steep learning curve. My main difficulty was navigating the multi-file architecture of the kernel to identify where changes were required. To

overcome this and streamline the learning process, I used AI (Gemini) as a guided instructional assistant to help explain kernel data structures, map file dependencies, and debug compiler errors. Through this process, I gained practical experience with kernel-level tracking and user-space system calls.
On a personal level, my mental headspace was the biggest hurdle during this assignment. Technical programming does not come naturally to me, and pushing myself to absorb complex low-level concepts is a real challenge. However, being in my final two semesters, staying focused on reaching the finish line kept me motivated to work through it.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).

Wait2() only retrieves metrics for the direct child process spawned by fork(). If the timed command spawns its own secondary child processes, time misses those CPU ticks undercounting total system resource usage.